

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 - Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	A.Sathwika Reddy	Reg. No.:	192525274
Section / Batch:	AIML	Date:	22/07/2026

Code of Conduct

I A.Sathwika Reddy-192525274 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A.Sathwika Reddy

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 - 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q1 Binary Search

5 marks

Identify the error in the following snippet.

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;  
    while(low < high) {  
        int mid = (low + high) / 2;  
        if(arr[mid] == key)  
            return mid;  
        else if(arr[mid] < key)  
            low = mid;  
        else  
            high = mid;  
    }  
    return -1;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int binarySearch(int arr[], int n, int key) {  
2     int low = 0, high = n - 1;  
3  
4     while(low <= high) { // Fix 1: Change '<' to '<='  
5         int mid = (low + high) / 2;  
6  
7         if(arr[mid] == key)  
8             return mid;  
9         else if(arr[mid] < key)  
10            low = mid + 1; // Fix 2: Move past the mid index  
11        else  
12            high = mid - 1; // Fix 3: Move before the mid index  
13    }  
14    return -1;  
15 }
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q2 Stack

5 marks

Find the error in the stack deletion operation. Justify it.

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int pop() {  
2     // Corrected underflow check  
3     if (top == -1) {  
4         printf("Underflow");  
5         return -1;  
6     }  
7     // Correctly returns current item, then decrements top  
8     return stack[top--];  
9 }  
10
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1 ✓
Binary Search
5 marks

Q2 ✓
Stack
5 marks

Q3 ✓
Stack
5 marks

Q4 ✓
Stack
5 marks

Q5 ✓
Stack
5 marks

Q6 ✓
Linked List
5 marks

Q7 ✓
Stack
5 marks

Q8 ✓
Linked List Data Structures
5 marks

Q9 ✓
Linked List
5 marks

Q10 ✓
Queue
5 marks

10/10 answered

Q3 Stack

5 marks

Check the snippet and identify the logical error.

```
int stack[5];
int top;
void init() {
    top = 0;
}
```

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int stack[5];
2 int top;
3
4 void init() {
5     top = -1; // Correctly initializes the stack as empty
6 }
7
```

Test Input

stdin input (optional)

Output

 Run

 Save

 Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1 ✓
Binary Search
5 marks

Q2 ✓
Stack
5 marks

Q3 ✓
Stack
5 marks

Q4 ✓
Stack
5 marks

Q5 ✓
Stack
5 marks

Q6 ✓
Linked List
5 marks

Q7 ✓
Stack
5 marks

Q8 ✓
Linked List Data Structures
5 marks

Q9 ✓
Linked List
5 marks

Q10 ✓
Queue
5 marks

10/10 answered

Q4 Stack

5 marks

Identify the errors in the following snippet.

```
if(ch == ')') {
while(stack[top] != ')') {
postfix[j++] = pop();
}
pop();
}
```

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if(ch == '(') {
2     while(stack[top] != '(') { // Changed ')' to '('
3         postfix[j++] = pop();
4     }
5     pop(); // Removes the '(' from the stack
6 }
7
```

Test Input

stdin input (optional)

Output

 Run

 Save

 Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1

Binary Search

5 marks



Q2

Stack

5 marks



Q3

Stack

5 marks



Q4

Stack

5 marks



Q5

Stack

5 marks



Q6

Linked List

5 marks



Q7

Stack

5 marks



Q8

Linked List Data Structures

5 marks



Q9

Linked List

5 marks



Q10

Queue

5 marks



10/10 answered

Q5 Stack

5 marks

In balancing Bracket, you identify the error.

```
if((ch == ']' && stack[top] == '[') ||  
(ch == '}' && stack[top] == '{')) {  
    pop();  
} else {  
    return 0;  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if ((ch == ']' && stack[top] == '[') ||  
2     (ch == '}' && stack[top] == '{') ||  
3     (ch == '(' && stack[top] == '(')) {  
4     pop();  
5 } else {  
6     return 0;  
7 }  
8
```

Run

Save

Submit Fix

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q6 Linked List

5 marks

What is the issue?

```
temp = head
while temp.next != NULL:
    print(temp.data)
temp = temp.next
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 temp = head
2 while temp != None: # Checks the node itself
3     print(temp.data)
4     temp = temp.next
5
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q7 Stack

5 marks

Point out the error.

```
POP(stack):  
if top == 0:  
    print("Underflow")  
else:  
    return stack[top--]
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 def pop(stack):  
2     global top # Required if top is a global variable tracking the index  
3  
4     # Check if stack is empty  
5     if top < 0:  
6         print("Underflow")  
7         return None  
8  
9     # Retrieve the item, decrement the index tracker, and return  
10    item = stack[top]  
11    top -= 1  
12    return item  
13
```



Run



Save



Submit Fix

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q8 Linked List Data Structures

5 marks

Identify the errors in the following code

```
while current != NULL:
    current.next = prev
    prev = current
    current = current.next
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 while current is not None:
2     nxt = current.next # 1. Save next node
3     current.next = prev # 2. Reverse the link
4     prev = current     # 3. Move prev forward
5     current = nxt      # 4. Move current forward
6
```

Test Input

stdin input (optional)

Output

 Run Save Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q9 Linked List

5 marks

What is the error in the below code?

```
new.next = head  
head = new
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 newNode.next = head  
2 head = newNode  
3
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks

Q2
Stack
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Stack
5 marks

Q6
Linked List
5 marks

Q7
Stack
5 marks


Q10 Queue

5 marks

What is missing in the below pseudocode?

DEQUEUE(Q):

if front == -1:

print("Underflow")

else:

return Q[front++]

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 DEQUEUE(Q):
2     if front == -1:
3         print("Underflow")
4     else:
5         element = Q[front]           // Store the current front element
6         if front == rear:           // Check if it was the last element
7             front = -1
8             rear = -1
9     else:
10        front = front + 1           // Move front to the next element
11    return element
```

Test Input

 stdin input (optional)


Output



CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

10/10 answered

Q1 Stack Notations - Postfix Expression

5 marks

Find the bug in the evaluation of a Postfix expression.

```
while (expression[i] != '\0') {  
    if (isdigit(expression[i])) push(expression[i] - '0');  
    else {  
        int op1 = pop();  
        int op2 = pop();  
        // perform operation: result = op1 + op2;  
        push(result);  
    }  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 while (expression[i] != '\0') {  
2     if (isdigit(expression[i])) {  
3         push(expression[i] - '0');  
4     }  
5     else {  
6         // First popped is the second operand  
7         int op2 = pop();  
8         // Second popped is the first operand  
9         int op1 = pop();  
10  
11         // Example logic for multiple operators  
12         int result;  
13         if (expression[i] == '+') result = op1 + op2;  
14         else if (expression[i] == '-') result = op1 - op2;  
15         else if (expression[i] == '*') result = op1 * op2;  
16         else if (expression[i] == '/') result = op1 / op2;  
17  
18         push(result);  
19     }  
20     // Bug Fix: Increment the index to avoid infinite loop  
21     i++;  
22 }  
23
```

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Stack
5 marks**Q9**
Circular Queue
5 marks**Q10**
Stack
5 marks**Q2** Circular Queue

5 marks

Identify the bug in this Circular Queue dequeue logic.

```
int dequeue() {  
    if (front == -1) return -1;  
    int data = queue[front];  
    front = (front + 1) % MAX;  
    return data;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int dequeue() {  
2     // 1. Check if queue is empty  
3     if (front == -1) return -1;  
4  
5     int data = queue[front];  
6  
7     // 2. Check if this is the last element  
8     if (front == rear) {  
9         front = -1;  
10        rear = -1;  
11    } else {  
12        // 3. Move front pointer circularly  
13        front = (front + 1) % MAX;  
14    }  
15  
16    return data;  
17 }  
18
```

Test Input

stdin input (optional)

Output

 Run Save Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks

Q2
Circular Queue
5 marks

Q3
Binary Search
5 marks

Q4
Queue
5 marks

Q5
Queue
5 marks

Q6
Doubly Linked List
5 marks

Q7
Stack
5 marks

Q8
Stack
5 marks

Q9
Circular Queue
5 marks

Q10
Stack
5 marks

Q3 Binary Search

5 marks

Identify the bug in this Recursive Binary Search.

```
int binarySearch(int arr[], int l, int r, int x) {  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == x) return mid;  
    if (arr[mid] > x) return binarySearch(arr, l, mid, x);  
    return binarySearch(arr, mid, r, x);  
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int binarySearch(int arr[], int l, int r, int x) {  
2     // Base Case: Element is not present  
3     if (l > r) return -1;  
4  
5     int mid = l + (r - l) / 2;  
6  
7     // Element found  
8     if (arr[mid] == x) return mid;  
9  
10    // Search left half  
11    if (arr[mid] > x) return binarySearch(arr, l, mid - 1, x);  
12  
13    // Search right half  
14    return binarySearch(arr, mid + 1, r, x);  
15 }  
16
```

Test Input

stdin input (optional)

Output

[▶ Run](#)[☁ Save](#)[🔧 Submit Fix](#)

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

Q4 Queue

5 marks

Find the error in this Deque insertFront operation.

```
void insertFront(int data) {  
    if (front == 0) front = MAX - 1;  
    else front = front - 1;  
    deque[front] = data;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 void insertFront(int data) {  
2     // 1. Check if the deque is full  
3     if ((front == 0 && rear == MAX - 1) || (front == rear + 1)) {  
4         cout << "Queue Overflow\n";  
5         return;  
6     }  
7  
8     // 2. Check if the deque is initially empty  
9     if (front == -1) {  
10        front = 0;  
11        rear = 0;  
12    }  
13    // 3. Handle circular wrap-around  
14    else if (front == 0) {  
15        front = MAX - 1;  
16    }  
17    // 4. Regular decrement  
18    else {  
19        front = front - 1;  
20    }  
21  
22    // Insert the element  
23    deque[front] = data;  
24 }  
25
```

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1 ✓
Stack Notations - Postfix Expression
5 marks

Q2 ✓
Circular Queue
5 marks

Q3 ✓
Binary Search
5 marks

Q4 ✓
Queue
5 marks

Q5 ✓
Queue
5 marks

Q6 ✓
Doubly Linked List
5 marks

Q7 ✓
Stack
5 marks

Q8 ✓

Q5 Queue

5 marks

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {
    if (rear == MAX) return;
    queue[rear++] = data;
}
```

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 void enqueue(int data) {
2     // Check if circular queue is full
3     if ((rear + 1) % MAX == front) {
4         printf("Queue Overflow\n");
5         return;
6     }
7
8     // Insert element and wrap around rear pointer
9     queue[rear] = data;
10    rear = (rear + 1) % MAX;
11 }
12
```

Test Input

Output

 Run

 Save

 Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q6 Doubly Linked List

5 marks

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {  
    if (del->prev != NULL)  
        del->prev->next = del->next;  
    if (del->next != NULL)  
        del->next->prev = del->prev;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 void deleteNode(Node* del) {  
2     // 1. Guard against NULL pointer  
3     if (del == NULL) return;  
4  
5     if (del->prev != NULL)  
6         del->prev->next = del->next;  
7  
8     if (del->next != NULL)  
9         del->next->prev = del->prev;  
10  
11     // 2. Free the allocated memory  
12     delete del;  
13 }  
14
```

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q7 Stack

5 marks

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 POP():  
2     if top == -1:                // Check for underflow  
3         print "Underflow Error"  
4         return Null  
5  
6     item = stack[top]            // Retrieve the top item  
7     top = top - 1                // Decrement the top pointer  
8     return item                  // Return the removed item  
9
```

Test Input

stdin input (optional)

Output



Run



Save



Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1 ✓
Stack Notations - Postfix
Expression
5 marks

Q2 ✓
Circular Queue
5 marks

Q3 ✓
Binary Search
5 marks

Q4 ✓
Queue
5 marks

Q5 ✓
Queue
5 marks

Q6 ✓
Doubly Linked List
5 marks

Q7 ✓
Stack
5 marks

Q8 Stack

5 marks

Point out the error.
if top == MAX:
Overflow

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if top == MAX - 1:
2     Overflow
3
```

Test Input

Output

 Run

 Save

 Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Stack
5 marks**Q9**
Circular Queue
5 marks**Q10****Q9 Circular Queue**

5 marks

Identify the issue in the following snippet for Circular Queue in Traffic Signal.

```
rear = (rear + 1) % MAX;  
queue[rear] = vehicle;  
if(rear == front){  
    printf("Queue Full");  
}
```

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 // 1. Check if the queue is full BEFORE inserting  
2 if ((rear + 1) % MAX == front) {  
3     printf("Queue Full\n");  
4 } else {  
5     // 2. Advance pointer and insert safely  
6     rear = (rear + 1) % MAX;  
7     queue[rear] = vehicle;  
8  
9     // Handle initial insertion if front is unset (-1)  
10    if (front == -1) {  
11        front = 0;  
12    }  
13 }  
14
```

Run

Save

Submit Fix

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1 ✓
Stack Notations - Postfix
Expression
5 marks

Q2 ✓
Circular Queue
5 marks

Q3 ✓
Binary Search
5 marks

Q4 ✓
Queue
5 marks

Q5 ✓
Queue
5 marks

Q6 ✓
Doubly Linked List
5 marks

Q10 Stack

5 marks

Point out the error?
PEEK(stack):
return stack[top+1]

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 PEEK(stack) :  
2     return stack[top]
```

Test Input

Output